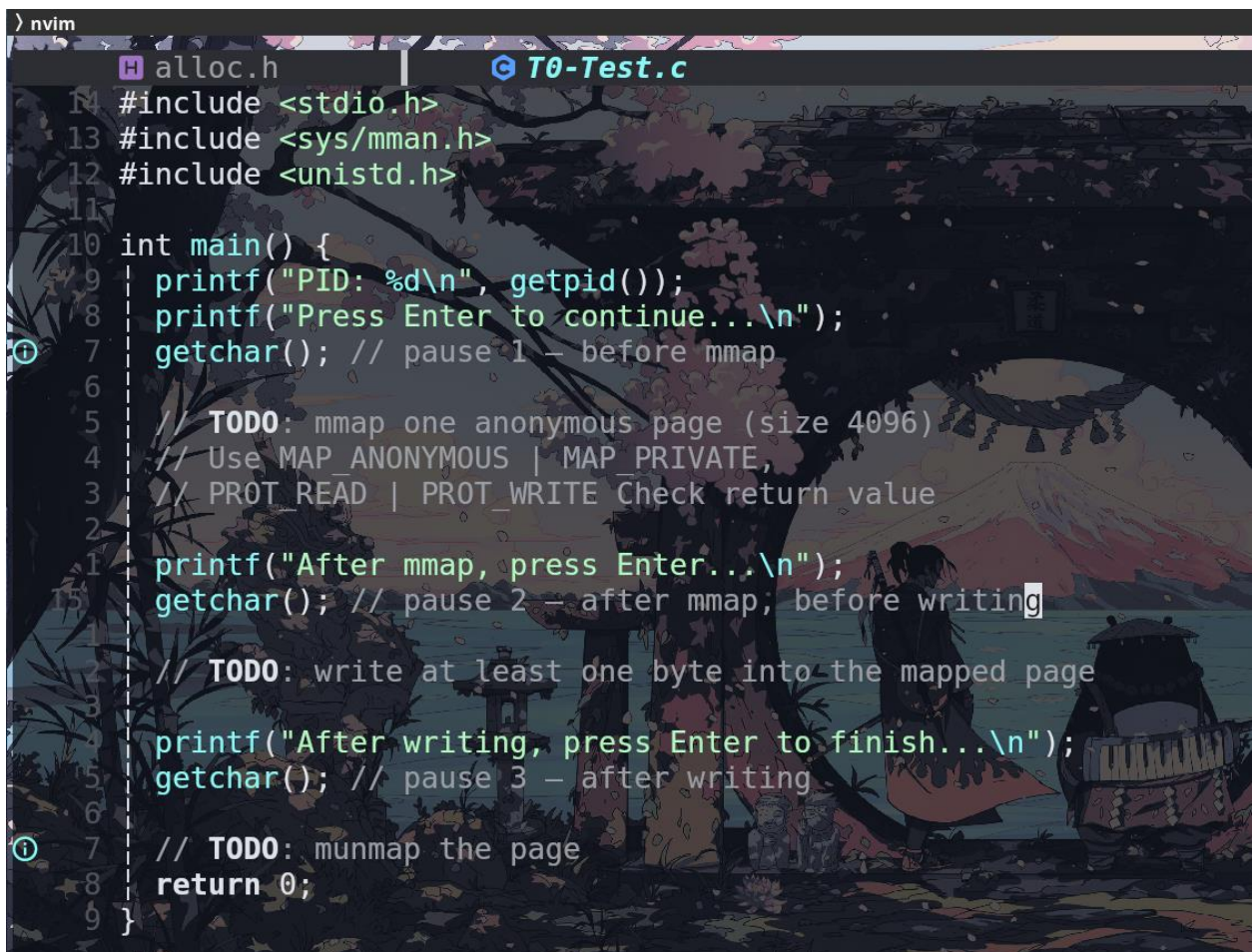


Assignment #2: Building your own Memory Manager

In this lab, you will understand the principles of memory management by building a custom memory manager to allocate memory dynamically in a program. Specifically, you will implement functions to allocate and free memory, that act as replacements for C library functions like malloc and free.

Task-1 (Exploring mmap)

1. T0_Test.c



```
nvim
alloc.h  T0-Test.c
14 #include <stdio.h>
13 #include <sys/mman.h>
12 #include <unistd.h>
11
10 int main() {
9     printf("PID: %d\n", getpid());
8     printf("Press Enter to continue...\n");
7     getchar(); // pause 1 - before mmap
6
5     // TODO: mmap one anonymous page (size 4096)
4     // Use MAP_ANONYMOUS | MAP_PRIVATE,
3     // PROT_READ | PROT_WRITE Check return value
2
1     printf("After mmap, press Enter...\n");
15    getchar(); // pause 2 - after mmap; before writing
1
2     // TODO: write at least one byte into the mapped page
3
4     printf("After writing, press Enter to finish...\n");
5     getchar(); // pause 3 - after writing
6
7     // TODO: munmap the page
8     return 0;
9 }
```

Skeleton for my T0-Test.c

```
> nvim
alloc.h | T0-Test.c
21 #include <sys/mman.h>
25 #include <unistd.h>
24
23 int main() {
22     printf("PID: %d\n", getpid());
21     printf("Press Enter to continue...\n");
20     getchar(); // Pause 1 – before mmap
19
18     // Map one anonymous page (4KB)
17     void *ptr = mmap(NULL, 4096, PROT_READ | PROT_WRITE,
16         MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
15
14     if (ptr == MAP_FILE) {
13         perror("mmap failed");
12         return 1;
11     }
10
9     printf("mmap succeeded at %p\n", ptr);
8     printf("Press Enter to continue (after mmpa, before write)...\n");
7     getchar(); // Pause 2
6
5     // Write one byte into the page
4     ((char *)ptr)[0] = 'A';
3     printf("Wrote to page\n");
2     printf("Press Enter to continue (after write)...\n");
1     getchar(); // Pause 3 – after writing
28
1     munmap(ptr, 4096);
2     printf("Unmapped, exiting.\n");
3     return 0;
4 }
```

```
./T0-Test
PID: 10593
Press Enter to continue...
mmap succeeded at 0x76c06a058000
Press Enter to continue (after mmpa, before write)...
Wrote to page
Press Enter to continue (after write)...
Unmapped, exiting.

Press ENTER or type command to continue
```

Seems to work.

```
~/cs4375-OS/alloc-code
iron@Ubuntu:~/cs4375-OS/alloc-code$ ./T0-Test
PID: 13198
Press Enter to continue...

mmap succeeded at 0x7c539fb54000
Press Enter to continue (after mmpa, before write)...

Wrote to page
Press Enter to continue (after write)...

Unmapped, exiting.
iron@Ubuntu:~/cs4375-OS/alloc-code$

iron@Ubuntu:~$ cat /proc/13198/status | grep -E "VmSize|VmRSS"
VmSize: 2912 kB
VmRSS: 1824 kB
iron@Ubuntu:~$ cat /proc/13198/status | grep -E "VmSize|VmRSS"
VmSize: 2916 kB
VmRSS: 1824 kB
iron@Ubuntu:~$ cat /proc/13198/status | grep -E "VmSize|VmRSS"
VmSize: 2916 kB
VmRSS: 1828 kB
iron@Ubuntu:~$ cat /proc/13198/status | grep -E "VmSize|VmRSS"
cat: /proc/13198/status: No such file or directory (os error 2)
iron@Ubuntu:~$
```

Before mmap we get the baseline virtual and physical memory, after mmap virtual size increased by 4KiB (RSS unchanged.), after write RSS increase by 4KiB, Virtual size remains the same.

Mmap reserves virtual address space but does not immediately allocate physical RAM (demand paging). The first write to the page triggers a page fault, causing the OS to assign a physical frame. This demonstrates the difference between virtual memory allocation and actual memory usage.

Task 2 – Building a memory manager in alloc.h

In this task, you will write code for a memory manager (alloc.h), to allocate and deallocate memory dynamically. Your memory manager must manage 4KB of memory, by requesting a 4KB page via mmap from the OS. You must support allocations and deallocations in sizes that are multiples of 8 bytes.

For data structure I decided to use a bit map.

```

16 #define PAGESIZE 4096 // size of memory to allocate from OS
15 #define MINALLOC 8    // allocations will be 8 bytes or multiples of it
14 #define GRANULE 8     // each granule is 8 bytes
13 #define NUM_GRANULES (PAGESIZE / GRANULE) // 512
12
11 // Global metadata
10 static void *page_base = NULL; // address of the 4KB page
9 static unsigned char bitmap[NUM_GRANULES / 8] = {0}; // 64 bytes
8 static int alloc_size[NUM_GRANULES] = {
7     0}; // size in bytes for each starting granule
6
5 // Check if a specific granule is free (bit = 0)
4 static int is_free(int idx) {
3     int byte = idx / 8;
2     int bit = idx % 8;
1     return !(bitmap[byte] & (1 << bit));
+ 26 }
+
1 // function declarations
+ 2 int init_alloc();
+ 3 int cleanup();
+ 4 char *alloc(int);
+ 5 void dealloc(char *);
+
+ 6
+ 7 #endif // ALLOC H
NORMAL main > +20 ~2 -1 alloc.h

```

I decided to create a helper function `is_free` that checks whether a specific granule in the 4KiB page is free. `Bitmap[byte]` is the byte holding the allocation status of 8 granules (one per bit), `(1 << bit)` creates a mask with a 1 only at the position of our granule, the bitwise AND yields a non-zero value if that bit is set, meaning that granule is allocated, and zero if it clear (free), the logical NOT turns zero (free) into 1(true) and non-zero into 0 (false).

```

+ 11 }
+ 10
+ 9 // Set or clear a range of granules
+ 8 static void set_range(int start, int count, int allocated) {
+ 7     for (int i = start; i < start + count; i++) {
+ 6         int byte = i / 8;
+ 5         int bit = i % 8;
+ 4         if (allocated)
+ 3             bitmap[byte] |= (1 << bit);
+ 2         else
+ 1             bitmap[byte] &= ~(1 << bit);
+
+ 1 }
+
+ 2

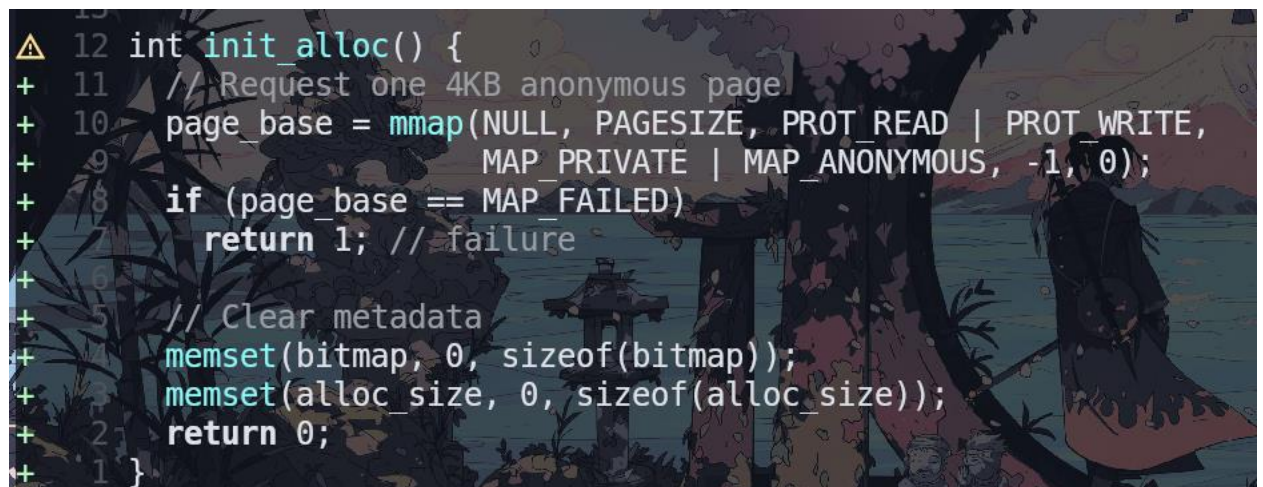
```

Without `set_range`, we would have to manually compute byte/bit indices and write OR/AND operations each time, increasing the chance of mistake, will make the code in `alloc` and `dealloc` simpler.



```
+ 5 // Find a run of at least 'needed' consecutive free granules (first fit)
+ 4 static int find_free_run(int needed) {
+ 3     int run_start = -1;
+ 2     int run_len = 0;
+ 1     for (int i = 0; i < NUM_GRANULES; i++) {
+ 45     if (is_free(i)) {
+ 1         if (run_len == 0)
+ 2             run_start = i;
+ 3             run_len++;
+ 4             if (run_start >= needed)
+ 5                 return run_start;
+ 6         } else {
+ 7             run_len = 0;
+ 8         }
+ 9     }
+ 10     return -1;
+ 11 }
```

Implements a first-fit strategy, it scans the granules from index 0 upwards and return the start of the first run that meets the length requirement. The scan is simple and ensures we always get the lowest possible address that can satisfy the request (first fit).



```
12 int init_alloc() {
+ 11     // Request one 4KB anonymous page
+ 10     page_base = mmap(NULL, PAGESIZE, PROT_READ | PROT_WRITE,
+ 9         MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
+ 8     if (page_base == MAP_FAILED)
+ 7         return 1; // failure
+ 6
+ 5     // Clear metadata
+ 4     memset(bitmap, 0, sizeof(bitmap));
+ 3     memset(alloc_size, 0, sizeof(alloc_size));
+ 2     return 0;
+ 1 }
```

Initialize the memory manager by obtaining a page from the OS and resetting metadata. The page is fresh and untouched, so the bitmap is entirely zero. No allocations have been made yet, so all granules are free.

```

10 int cleanup() {
11     if (page_base) {
12         if (munmap(page_base, PAGE_SIZE) == -1)
13             return 1;
14         page_base = NULL;
15     }
16     // metadata is static will be overwritten on the next init_alloc
17     return 0;
18 }

```

Release the page back to the OS and clean up any resources. Since all metadata is static, it will be overwritten. The metadata is not freed because it is statically allocated, it will be reused when `init_alloc` is called again. The OS reclaims the physical page.

```

17
18 char *alloc(int size) {
19     // Validate request
20     if (size <= 0 || size % MINALLOC != 0 || size > PAGE_SIZE)
21         return NULL;
22     int needed = size / GRANULE; // number of granules required
23     int start = find_free_run(needed);
24     if (start == -1)
25         return NULL; // not enough contiguous free space
26     // Mark granules as allocated
27     set_range(start, needed, 1);
28     // Record the size at the starting granule
29     alloc_size[start] = size;
30     return (char *)page_base + start * GRANULE;
31 }
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97

```

Allocate a block of memory of the request size (must be a multiple of 8) and return a pointer to it, or NULL if the request cannot be satisfied. The bitmap ensures that no two allocations overlap. The `alloc_size` array preserves the exact size of the block, which may be less than the full run length if the free run was larger. This size is later used to free exactly the correct number of granules.

```

8 void dealloc(char *ptr) {
7     if (!ptr || !page_base)
6         return;
5
4     // Compute granule index
3     int idx = (ptr - (char *)page_base) / GRANULE;
2     if (idx < 0 || idx >= NUM_GRANULES)
1         return; // pointer out of range
6
1     int size = alloc_size[idx];
2     if (size == 0) {
3         return; // not a valid allocated block
4     }
6
7     int count = size / GRANULE; // number of granules to free
8
9     // Clear the bits in the bitmap
10    set_range(idx, count, 0);
11    // Reset the size entry
12    alloc_size[idx] = 0;
13 }

```

Free a previously allocated block so that its granules become available for future allocations. The bitmap and alloc_size together maintain the integrity of the free space. Freeing exactly the right granules and resetting the size allows the block to be reused and coalesced automatically.

First test run had error and I use some print statement to track it down.

```

~/cs4375-OS/alloc-code
iron@Ubuntu:~/cs4375-OS/alloc-code$ ./testalloc
no suitable run found-1no suitable run found-1(null) (null)alloc failed
iron@Ubuntu:~/cs4375-OS/alloc-code$

```

This error is in the find_free_run

```

12 #include "alloc.h"
11 #include <stdio.h>
10 #include <string.h>
9 #include <sys/mman.h>
8 #include <unistd.h>

6 int main() {
5 // mmap to get page
4 if (init_alloc())
3 return 1; // mmap failed

2
1 char *str = alloc(512);
13 char *str2 = alloc(512);
1 if (str == NULL || str2 == NULL) {
2 printf("alloc failed\n");
3 return 1;
4 }

5 strcpy(str, "Hello, world!");
6 if (strcmp("Hello, world!", str))
7 printf("Hello, world! test failed\n");
8
9 else
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

```

9 // Find a run of at least 'needed' consecutive free granules (first fit)
8 static int find_free_run(int needed) {
7 int run_start = -1;
6 int run_len = 0;
5 for (int i = 0; i < NUM_GRANULES; i++) {
4 if (is_free(i)) { // granule i is free
3 if (run_len == 0) // start of a new run
2 run_start = i;
1 run_len++;
0 if (run_start >= needed) // Found the error :) run_start is not suppose to be there.
1 return run_start;
2 } else {
3 run_len = 0;
4 }
5 }
6 printf("no suitable run found");
7 return -1;
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

While checking logic of the function and flow I realize that I had a simple error.

```

9 // Find a run of at least 'needed' consecutive free granules (first fit)
8 static int find_free_run(int needed) {
7 int run_start = -1;
6 int run_len = 0;
5 for (int i = 0; i < NUM_GRANULES; i++) {
4 if (is_free(i)) { // granule i is free
3 if (run_len == 0) // start of a new run
2 run_start = i;
1 run_len++; // extend current run
0 if (run_len >= needed) // found a run long enough
1 return run_start;
2 } else { // granule i is allocated
3 run_len = 0; // reset run
4 }
5 }
6 return -1; // no suitable run found
7 }
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

Fixed code for find_free_run

```
iron@Ubuntu:~/cs4375-0S/alloc-code$ ./testalloc
Hello, world! test passed
Elementary tests passed
Starting comprehensive tests (see details in code)
Test 1 passed: allocated 4 chunks of 1KB each
Test 2 passed: dealloc and realloc worked
Test 3 passed: dealloc and smaller realloc worked
Test 4 passed: merge worked
Test 5 passed: merge alloc 2048 worked
iron@Ubuntu:~/cs4375-0S/alloc-code$ ]
```

All Test passed after fix.

```
> nvim
mytest.c
1 #include "alloc.h"
2 #include <stdio.h>
3 #include <string.h>
4
5 int main() {
6     printf("--- Custom Memory Manager Test ---\n");
7     if (init_alloc()) {
8         printf("init_alloc failed, aborting.\n");
9         return 1;
10    }
11 }
```

Since we need to create more test, I created a mytest.c file

```
6 // macro helper to print test result
5 #define TEST(cond, msg)
4     do {
3         if (cond)
2             printf("[PASS] %s\n", msg);
1         else
|             printf("[FAIL] %s\n", msg);
1     } while (0)
2
```

```
+ 15 // Test 1: Basic allocation and deallocation
+ 14 printf("--- Test 1: Basic alloc/dealloc --- \n");
+ 13
+ 12 char *p1 = alloc(8);
+ 11 char *p2 = alloc(16);
+ 10 char *p3 = alloc(24);
+ 9
+ 8 TEST(p1 != NULL && p2 != NULL && p3 != NULL, "Three allocations successful");
+ 7 TEST(p1 + 8 == p2 || p2 + 16 == p3, "Pointer are contiguous (likely)");
+ 6
+ 5 dealloc(p2);
+ 4
+ 3 char *p4 = alloc(16);
+ 2
+ 1 TEST(p4 == p2, "Reuse freed block (first fit)");
+ 37
+ 1 dealloc(p1);
+ 2 dealloc(p3);
+ 3 dealloc(p4);
+ 4
+ 5 printf("\n");
+ 6 }
```

```
// Test 2: Full page allocations
printf("--- Test 2: Full page allocation --- \n");

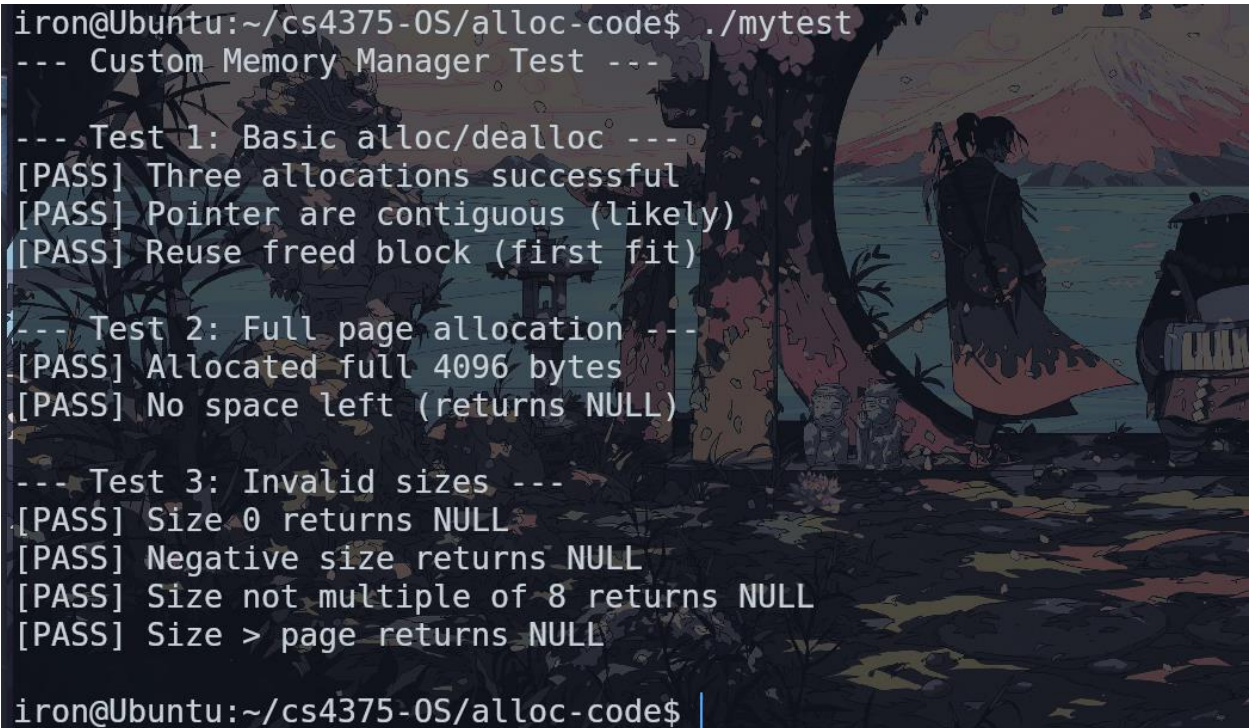
char *big = alloc(4096);

TEST(big != NULL, "Allocated full 4096 bytes");

char *should_fail = alloc(8);

TEST(should_fail == NULL, "No space left (returns NULL)");
dealloc(big);
printf("\n");
```

```
// Test 3: Invalid sizes
printf("--- Test 3: Invalid sizes ---\n");
TEST(alloc(0) == NULL, "Size 0 returns NULL");
TEST(alloc(-8) == NULL, "Negative size returns NULL");
TEST(alloc(7) == NULL, "Size not multiple of 8 returns NULL");
TEST(alloc(4100) == NULL, "Size > page returns NULL");
printf("\n");
```

A terminal window showing the execution of a custom memory manager test. The background is a dark, artistic illustration of a person in traditional Japanese clothing standing by a body of water with a mountain in the distance. The text is white on a dark background.

```
iron@Ubuntu:~/cs4375-0S/alloc-code$ ./mytest
--- Custom Memory Manager Test ---

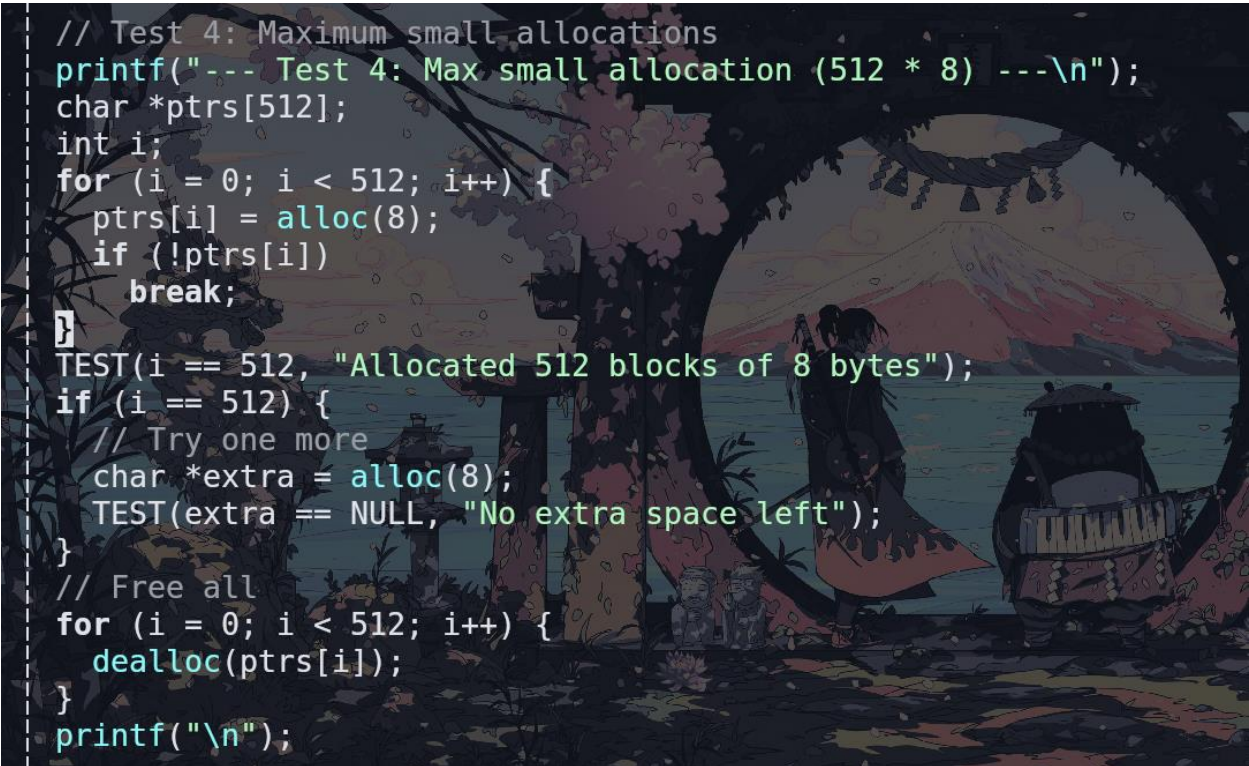
--- Test 1: Basic alloc/dealloc ---
[PASS] Three allocations successful
[PASS] Pointer are contiguous (likely)
[PASS] Reuse freed block (first fit)

--- Test 2: Full page allocation ---
[PASS] Allocated full 4096 bytes
[PASS] No space left (returns NULL)

--- Test 3: Invalid sizes ---
[PASS] Size 0 returns NULL
[PASS] Negative size returns NULL
[PASS] Size not multiple of 8 returns NULL
[PASS] Size > page returns NULL

iron@Ubuntu:~/cs4375-0S/alloc-code$ |
```

So far so good.

A code editor window showing C code for Test 4. The background is the same dark, artistic illustration as the terminal window. The code is color-coded: comments are green, strings are green, and code is white.

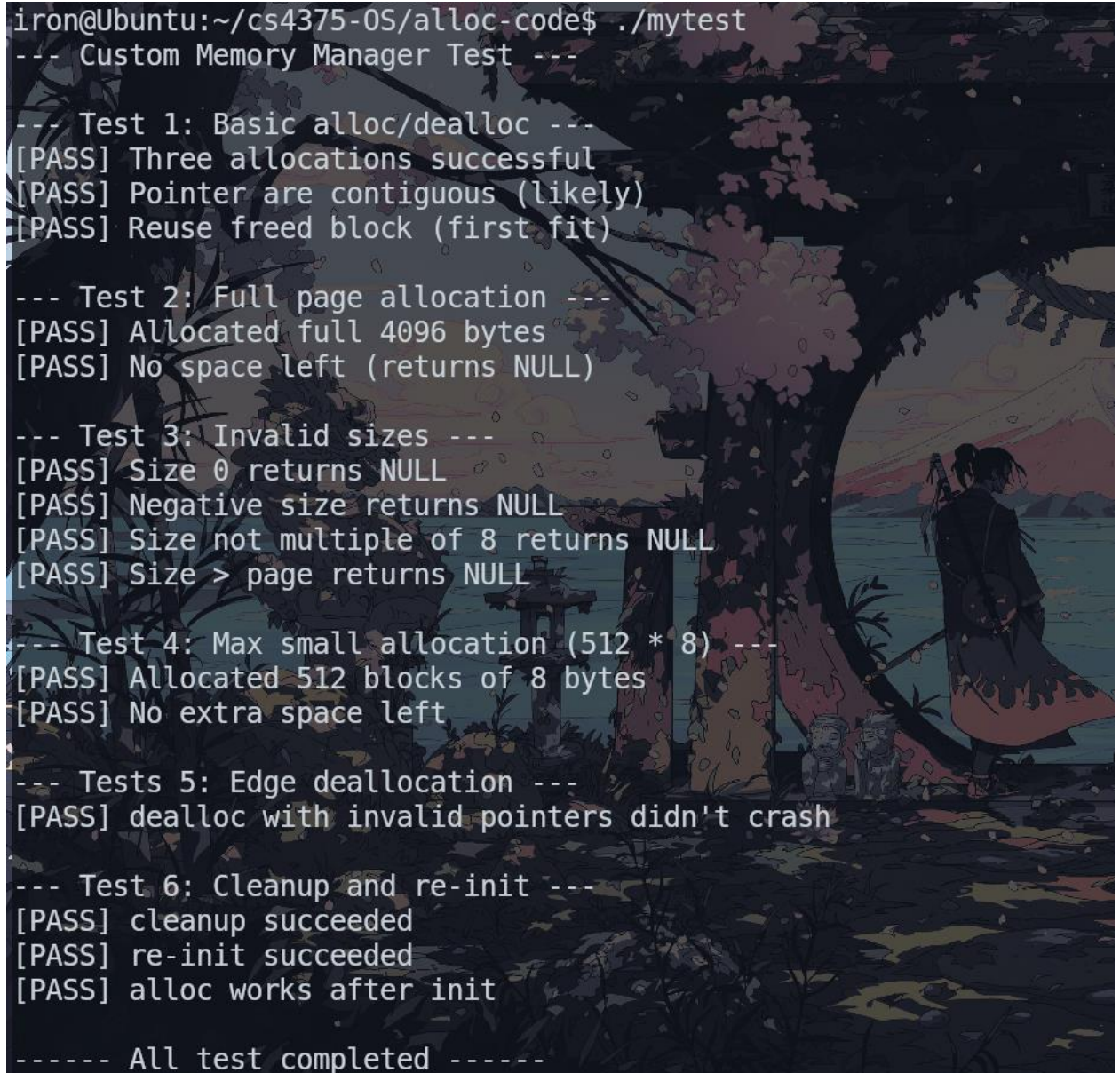
```
// Test 4: Maximum small allocations
printf("--- Test 4: Max small allocation (512 * 8) ---\n");
char *ptrs[512];
int i;
for (i = 0; i < 512; i++) {
    ptrs[i] = alloc(8);
    if (!ptrs[i])
        break;
}
TEST(i == 512, "Allocated 512 blocks of 8 bytes");
if (i == 512) {
    // Try one more
    char *extra = alloc(8);
    TEST(extra == NULL, "No extra space left");
}
// Free all
for (i = 0; i < 512; i++) {
    dealloc(ptrs[i]);
}
printf("\n");
```

```
--- Test 4: Max small allocation (512 * 8) ---  
[PASS] Allocated 512 blocks of 8 bytes  
[PASS] No extra space left
```

```
// Test 5: Dealloc with NULL or invalid pointer (should not crash)  
printf("--- Tests 5: Edge deallocation ---\n");  
dealloc(NULL);  
dealloc((char *)0x12345678); // likely out of range, should be ignored.  
TEST(1, "dealloc with invalid pointers didn't crash");  
printf("\n");
```

```
--- Tests 5: Edge deallocation ---  
[PASS] dealloc with invalid pointers didn't crash
```

```
// Test 6: Cleanup and re-initialize  
printf("--- Test 6: Cleanup and re-init ---\n");  
TEST(cleanup() == 0, "cleanup succeeded");  
TEST(init_alloc() == 0, "re-init succeeded");  
  
char *test = alloc(8);  
TEST(test != NULL, "alloc works after init");  
dealloc(test);  
cleanup();  
printf("\n");  
  
printf("----- All test completed -----\n");  
return 0;  
}
```



```
iron@Ubuntu:~/cs4375-OS/alloc-code$ ./mytest
--- Custom Memory Manager Test ---

--- Test 1: Basic alloc/dealloc ---
[PASS] Three allocations successful
[PASS] Pointer are contiguous (likely)
[PASS] Reuse freed block (first fit)

--- Test 2: Full page allocation ---
[PASS] Allocated full 4096 bytes
[PASS] No space left (returns NULL)

--- Test 3: Invalid sizes ---
[PASS] Size 0 returns NULL
[PASS] Negative size returns NULL
[PASS] Size not multiple of 8 returns NULL
[PASS] Size > page returns NULL

--- Test 4: Max small allocation (512 * 8) ---
[PASS] Allocated 512 blocks of 8 bytes
[PASS] No extra space left

--- Tests 5: Edge deallocation ---
[PASS] dealloc with invalid pointers didn't crash

--- Test 6: Cleanup and re-init ---
[PASS] cleanup succeeded
[PASS] re-init succeeded
[PASS] alloc works after init

----- All test completed -----
```

Reference code: <https://github.com/ilke-kas/dma-memory-allocator-OS-project/blob/main/dma.c>

First fit: <https://github.com/muthuvelsanthakumar/IMPLEMENTATION-OF-MEMORY-MANAGEMENT-SCHEMES/blob/main/IMPLEMENTATION%20OF%20MEMORY%20MANAGEMENT%20SCHEMES>

https://man7.org/tlpi/code/online/dist/mmap/anon_mmap.c.html

Chapter 13 – 20 from OS Three Easy Pieces.